

5CS022 Distribute and Cloud Systems Programming

Week 1 Workshop

Overview

The aim of this workshop is to familiarise you with building, compiling and running MPI programs. You can carry out this workshop on your own Linux system

Tasks

1. Download the sample MPI programs from the drive into your Linux system. Compile and run the program `mpi01.c`. To compile it, run the following command in the terminal:

```
mpicc mpi01.c -o mpi01
```

Now run it with the following:

```
mpiexec ./mpi01
```

This will (probably) only run only one process, which is not very interesting. Run it again with the following command::

```
mpiexec -n 4 ./mpi01
```

Note the output this time. It should indicate that 4 processes have run and they all have different process IDs.

Experiment with higher and higher numbers of processes until it stops running. Then have a look at the error message and try to work out why it stopped working.\

Output:

```
ojashshakya@LenovoL0Q:/mnt/c/Users/Lenovo/OneDrive - University of Wolverhampton/2nd Year - 4th Sem/Distributed and Cloud Systems Programming/Week 1$ mpicc mpi01.c -o mpi01
ojashshakya@LenovoL0Q:/mnt/c/Users/Lenovo/OneDrive - University of Wolverhampton/2nd Year - 4th Sem/Distributed and Cloud Systems Programming/Week 1$ mpiexec -n 4 ./mpi01
I am 0 of 1
I am 0 of 1
I am 0 of 1
I am 0 of 1
I am 0 of 1
ojashshakya@LenovoL0Q:/mnt/c/Users/Lenovo/OneDrive - University of Wolverhampton/2nd Year - 4th Sem/Distributed and Cloud Systems Programming/Week 1$ mpiexec -n 3 ./mpi01
I am 0 of 1
I am 0 of 1
I am 0 of 1
ojashshakya@LenovoL0Q:/mnt/c/Users/Lenovo/OneDrive - University of Wolverhampton/2nd Year - 4th Sem/Distributed and Cloud Systems Programming/Week 1$ mpiexec -n 300 ./mpi01
[proxy:0@LenovoL0Q] HYDU_create_process (lib/utils/launch.c:24): pipe error (Too many open files)
[proxy:0@LenovoL0Q] launch_procs (proxy/pmip_cb.c:1000): create process returned error
[proxy:0@LenovoL0Q] handle_launch_procs (proxy/pmip_cb.c:585): launch_procs returned error
[proxy:0@LenovoL0Q] HYD_pmcd_pmip_control_cmd_cb (proxy/pmip_cb.c:495): launch_procs returned error
[proxy:0@LenovoL0Q] HYDT_dmxu_poll_wait_for_event (lib/tools/demux/demux_poll.c:76): callback returned error status
[proxy:0@LenovoL0Q] main (proxy/pmip.c:122): demux engine error waiting for event
[mpiexec@LenovoL0Q] control_cb (mpiexec/pmiserv_cb.c:280): assert (!closed) failed
[mpiexec@LenovoL0Q] HYDT_dmxu_poll_wait_for_event (lib/tools/demux/demux_poll.c:76): callback returned error status
[mpiexec@LenovoL0Q] HYD_pmci_wait_for_completion (mpiexec/pmiserv_pmci.c:173): error waiting for event
[mpiexec@LenovoL0Q] main (mpiexec/mpiexec.c:260): process manager error waiting for completion
ojashshakya@LenovoL0Q:/mnt/c/Users/Lenovo/OneDrive - University of Wolverhampton/2nd Year - 4th Sem/Distributed and Cloud Systems Programming/Week 1$
```

Explanation:

Here, the mpicc command compiles the program using MPI by creating an executable file mpi01. Then, mpiexec command runs the program which the first runs only on process by default and the second run uses -n to specify number for process i.e. 4 which means the program will be separated into 4 processes.

2. Compile and run the program mpi02.c. Try running it with 2, 3 and 4 processes. Eg.:

```
mpiexec -n 2 ./mpi02
```

```
mpiexec -n 3 ./mpi02
```

```
mpiexec -n 4 ./mpi02
```

Note what happens. It doesn't let you run the program with anything other than 3 processes.

Output:

```
ojashshakya@LenovoL0Q:/mnt/c/Users/Lenovo/OneDrive - University of Wolverhampton/2nd Year - 4th Sem/Distributed and Cloud Systems Programming/Week 1$ nano
ojashshakya@LenovoL0Q:/mnt/c/Users/Lenovo/OneDrive - University of Wolverhampton/2nd Year - 4th Sem/Distributed and Cloud Systems Programming/Week 1$ mpicc mpi02.c -o mpi02
ojashshakya@LenovoL0Q:/mnt/c/Users/Lenovo/OneDrive - University of Wolverhampton/2nd Year - 4th Sem/Distributed and Cloud Systems Programming/Week 1$ mpiexec -n 2 ./mpi02
This program needs to run on exactly 3 processes
ojashshakya@LenovoL0Q:/mnt/c/Users/Lenovo/OneDrive - University of Wolverhampton/2nd Year - 4th Sem/Distributed and Cloud Systems Programming/Week 1$ mpiexec -n 3 ./mpi02
Process 2 received 17
Process 1 received 9
ojashshakya@LenovoL0Q:/mnt/c/Users/Lenovo/OneDrive - University of Wolverhampton/2nd Year - 4th Sem/Distributed and Cloud Systems Programming/Week 1$ mpiexec -n 4 ./mpi02
This program needs to run on exactly 3 processes
ojashshakya@LenovoL0Q:/mnt/c/Users/Lenovo/OneDrive - University of Wolverhampton/2nd Year - 4th Sem/Distributed and Cloud Systems Programming/Week 1$ |
```

Explanation:

In this program, while running mpi02.c, it only works correctly when exactly 3 processes are used in which size refers to the number of process obtained by MPI_Comm_size(). Executing this program with 2 or 4 processes will give an error message as "This program needs to run on exactly 3 processes" as 3 processes will execute without any error.

- Now change the code so that you remove the check for only 3 processes. Now run it with 2, then 3, then 4 and then more processes.

Program:

```
GNU nano 7.2 mpi02.c
/*
- Process 0 sends x to process 1 and y to processes 2. "1, MPI_INT"
  indicates that the message consists of one integer.
- Processes other than rank 0 wait to receive a message using MPI_Recv.
  The "0, 0" indicates that the message is expected from process 0 and
  should have the tag 0. The result is stored in the number variable.
*/

#include <stdio.h>
#include <mpi.h>

int main(int argc, char** argv) {
    int size, rank;

    MPI_Init(NULL, NULL);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    if(rank ==0){
        int x = 9;
        int y = 17;
        MPI_Send(&x, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
        MPI_Send(&y, 1, MPI_INT, 2, 0, MPI_COMM_WORLD);
    } else {
        int number;
        MPI_Recv(&number, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        printf("Process %d received %d\n", rank, number);
    }
    MPI_Finalize();

    return 0;
}

ojashshakya@LenovoL0Q:/mnt/c/Users/Lenovo/OneDrive - University of Wolverhampton/2nd Year - 4th Sem/Distributed and Cloud Systems Programming/Week 1$ mpicc mpi02.c -o mpi02
ojashshakya@LenovoL0Q:/mnt/c/Users/Lenovo/OneDrive - University of Wolverhampton/2nd Year - 4th Sem/Distributed and Cloud Systems Programming/Week 1$ mpiexec -n 2 --oversubscribe ./mpi02
Process 1 received 9
[LenovoL0Q:04444] *** An error occurred in MPI_Send
[LenovoL0Q:04444] *** reported by process [3925278721,0]
[LenovoL0Q:04444] *** on communicator MPI_COMM_WORLD
[LenovoL0Q:04444] *** MPI_ERR_RANK: invalid rank
[LenovoL0Q:04444] *** MPI_ERRORS_ARE_FATAL (processes in this communicator will now abort,
[LenovoL0Q:04444] *** and potentially your MPI job)
ojashshakya@LenovoL0Q:/mnt/c/Users/Lenovo/OneDrive - University of Wolverhampton/2nd Year - 4th Sem/Distributed and Cloud Systems Programming/Week 1$ mpiexec -n 3 --oversubscribe ./mpi02
Process 2 received 17
Process 1 received 9
ojashshakya@LenovoL0Q:/mnt/c/Users/Lenovo/OneDrive - University of Wolverhampton/2nd Year - 4th Sem/Distributed and Cloud Systems Programming/Week 1$ mpiexec -n 4 --oversubscribe ./mpi02
Process 1 received 9
Process 2 received 17
^C
ojashshakya@LenovoL0Q:/mnt/c/Users/Lenovo/OneDrive - University of Wolverhampton/2nd Year - 4th Sem/Distributed and Cloud Systems Programming/Week 1$
```

Explanation:

When the condition to check for only 3 process is removed and allowed running on any number of processes, the program can now run on any number of processes with 2, 3, 4 or more processes without errors. But, when we try to run it with 4 or more processes, the programs appear to run but never ends, we need to force end the program.

4. When you try to run it with 4 or more processes, it probably runs and appears to work but never ends. You will have to end with "Ctrl-C". Why do you think it doesn't end when you run it with more than 3 processes? Change it so that it will work with any number of processes.

Program:

```
GNU nano 7.2 mpi02.c
/*
- Process 0 sends x to process 1 and y to processes 2. "1, MPI_INT"
  indicates that the message consists of one integer.
- Processes other than rank 0 wait to receive a message using MPI_Recv.
  The "0, 0" indicates that the message is expected from process 0 and
  should have the tag 0. The result is stored in the number variable.
*/

#include <stdio.h>
#include <mpi.h>

int main(int argc, char** argv) {
    int size, rank;

    MPI_Init(NULL, NULL);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    if (rank == 0) {
        for (int i = 1; i < size; i++) {
            int num = 10 + i;
            MPI_Send(&num, 1, MPI_INT, i, 0, MPI_COMM_WORLD);
        }
    }
    else {
        int number;
        MPI_Recv(&number, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        printf("Process %d received %d from process 0\n", rank, number);
    }

    MPI_Finalize();
    return 0;
}
```

```
ojashshakya@LenovoL0Q:/mnt/c/Users/Lenovo/OneDrive - University of Wolverhampton/2nd Year - 4th Sem/Distributed and Cloud Systems Programming/Week 1$ mpiexec -n 2 ./mpi02
Process 1 received 11 from process 0
ojashshakya@LenovoL0Q:/mnt/c/Users/Lenovo/OneDrive - University of Wolverhampton/2nd Year - 4th Sem/Distributed and Cloud Systems Programming/Week 1$ mpiexec -n 3 ./mpi02
Process 2 received 12 from process 0
Process 1 received 11 from process 0
ojashshakya@LenovoL0Q:/mnt/c/Users/Lenovo/OneDrive - University of Wolverhampton/2nd Year - 4th Sem/Distributed and Cloud Systems Programming/Week 1$ mpiexec -n 4 ./mpi02
Process 2 received 12 from process 0
Process 3 received 13 from process 0
Process 1 received 11 from process 0
ojashshakya@LenovoL0Q:/mnt/c/Users/Lenovo/OneDrive - University of Wolverhampton/2nd Year - 4th Sem/Distributed and Cloud Systems Programming/Week 1$ mpiexec -n 6 ./mpi02
Process 4 received 14 from process 0
Process 5 received 15 from process 0
Process 1 received 11 from process 0
Process 2 received 12 from process 0
Process 3 received 13 from process 0
ojashshakya@LenovoL0Q:/mnt/c/Users/Lenovo/OneDrive - University of Wolverhampton/2nd Year - 4th Sem/Distributed and Cloud Systems Programming/Week 1$ mpiexec --oversubscribe -n 10 ./mpi02
Process 3 received 13 from process 0
Process 4 received 14 from process 0
Process 5 received 15 from process 0
Process 6 received 16 from process 0
Process 7 received 17 from process 0
Process 8 received 18 from process 0
Process 9 received 19 from process 0
Process 1 received 11 from process 0
Process 2 received 12 from process 0
ojashshakya@LenovoL0Q:/mnt/c/Users/Lenovo/OneDrive - University of Wolverhampton/2nd Year - 4th Sem/Distributed and Cloud Systems Programming/Week 1$
```

Explanation:

While running with 4 or more processes the program doesn't stop and it hangs. It happens due to blocking communication, where the processes waiting for message from others that never get received. In MPI, if one process expects input from a specific source but never receives it, the program waits indefinitely which results in program to never end.

5. Build and run the program mpi03.c. In this program Process 0 will wait for messages from Process 1 and Process 2. However, Process 1 ends up blocking Process 2 because it sleeps for 5 seconds.

Output:

```
ojashshakya@LenovoL0Q:/mnt/c/Users/Lenovo/OneDrive - University of Wolverhampton/2nd Year - 4th Sem/Distributed and Cloud Systems Programming/Week 1$ nano mpi03.c
ojashshakya@LenovoL0Q:/mnt/c/Users/Lenovo/OneDrive - University of Wolverhampton/2nd Year - 4th Sem/Distributed and Cloud Systems Programming/Week 1$ mpicc mpi03.c -o mpi03
ojashshakya@LenovoL0Q:/mnt/c/Users/Lenovo/OneDrive - University of Wolverhampton/2nd Year - 4th Sem/Distributed and Cloud Systems Programming/Week 1$ mpiexec -n 3 ./mpi03
Received 11 from process 1
Received 12 from process 2
ojashshakya@LenovoL0Q:/mnt/c/Users/Lenovo/OneDrive - University of Wolverhampton/2nd Year - 4th Sem/Distributed and Cloud Systems Programming/Week 1$ |
```

Explanation:

In this program, the process 0 waits for messages from both process 1 and process 2. However, process 1 sleeps for 5 seconds before sending its message using MPI_Send(), while process 2 sends its message immediately. As a result, Process 0 receives Process 2's message right away but remains blocked, waiting for process 1. This happens because MPI_Recv() is a synchronous, blocking function, meaning Process 0 cannot proceed until all expected messages arrive. This illustrates how delays in one process can cause others to stall, showcasing MPI's blocking behaviour.

6. The following is a simple program that looks for prime numbers between 1 to 10000:

```
#include <stdio.h>
```

```
int main(int argc, char **argv)
{
    int i, c;
    int nstart=1, nfinish=10000;
    printf("%s : Prime numbers between %d and %d are :\n",
        argv[0], nstart, nfinish);
    for(i=nstart; i<=nfinish; i++)
    {
        for(c=2; c<=i-1; c++)
        {
            if ( i%c==0 )
                break;
        }
        if ( c==i )
            printf("%s : %d\n",argv[0], i);
    }
    return 0;
}
```

Convert it to MPI so that it can run with different numbers of processes including just one process.

Program:

```
GNU nano 7.2 mpi04.c
#include <stdio.h>
#include <mpi.h>

int is_prime(int num) {
    if (num < 2) return 0;
    for (int i = 2; i * i <= num; i++) {
        if (num % i == 0) return 0;
    }
    return 1;
}

int main(int argc, char **argv) {
    int rank, size;
    int nstart = 1, nfinish = 10000;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    int chunk_size = (nfinish - nstart + 1) / size;
    int start = nstart + rank * chunk_size;
    int end = (rank == size - 1) ? nfinish : start + chunk_size - 1;

    printf("Process %d checking numbers from %d to %d\n", rank, start, end);
    for (int i = start; i <= end; i++) {
        if (is_prime(i)) {
            printf("Process %d: %d is prime\n", rank, i);
        }
    }

    MPI_Finalize();
    return 0;
}

ojashshakya@LenovoL0Q:/mnt/c/Users/Lenovo/OneDrive - University of Wolverhampton/2nd Year - 4th Sem/Distributed and Cloud Systems Programming/Week 1$ mpicc mpi04.c -o mpi04
ojashshakya@LenovoL0Q:/mnt/c/Users/Lenovo/OneDrive - University of Wolverhampton/2nd Year - 4th Sem/Distributed and Cloud Systems Programming/Week 1$ mpiexec -n 3 ./mpi04
Process 1 checking numbers from 3334 to 6666
Process 1: 3343 is prime
Process 1: 3347 is prime
Process 1: 3359 is prime
Process 1: 3361 is prime
Process 1: 3371 is prime
Process 1: 3373 is prime
Process 2: 9949 is prime
Process 2: 9967 is prime
Process 2: 9973 is prime
ojashshakya@LenovoL0Q:/mnt/c/Users/Lenovo/OneDrive - University of Wolverhampton/2nd Year - 4th Sem/Distributed and Cloud Systems Programming/Week 1$ |
```

Explanation:

This program converts a sequential prime number finder into a parallel program using MPI. The original code iterates through numbers from `nstart` to `nfinish` (1 to 10,000) and checks if each number is prime by testing divisibility. To parallelize this, the range is divided among processes, so each process calculates primes within its assigned chunk. Here, `size` is the total number of processes, and `rank` is the unique ID of each process. Each process independently iterates through its range, checking for primes. The results, along with the process rank are displayed.